

بناء المترجمات

إعداد: د.م. عبد العظيم عمّوري

في الجلسة السابقة (الخامسة عشرة)، نجحنا في بناء الأساسيات الرياضية وعلّمنا المترجم كيف يفكّك المعادلات إلى تعليمات متسلسلة، ونجحنا في تخطي قانون التعيين الفردي (SSA) للتعامل مع المتغيرات القابلة للتعديل عبر مكّس الذاكرة. لكن، البرنامج حتى هذه اللحظة يعمل كخط مستقيم أعمى.

اليوم سنواجه التحدي الهندسي الممتع: إعطاء لغتنا "العقل المدبر". هدفنا هو برمجة توليد الكود الوسيط (IR) لجملّة "إذا" وتطبيق القفزات الشرطية.

إليك الدليل العملي خطوة بخطوة. تأكدوا من تفعيل بيئة بايثون الافتراضية (venv) واستعدوا لهندسة مسارات التحكم.

الدليل العملي - الجلسة السادسة عشرة: برمجة مسارات التحكم (الجمل الشرطية)

الخطوة الأولى: التخطيط الهندسي لمسارات التحكم (Control Flow Graph)

كما تعلّمنا في المحاضرة النظرية، لغة LLVM IR لا تفهم الأقواس أو النطاقات، بل تعتمد كلياً على القفزات بين الكتل. عندما نعثر على جملة شرطية، يجب أن نشق طريق التنفيذ إلى مسارين، ثم نجمعهما مجدداً. لذلك، سنقوم بتعليم زائرنا كيف ينشئ ثلاث كتل أساسية في الذاكرة:

1. كتلة التحقق (Then): تنفذ إذا كان الشرط صحيحاً.

2. كتلة النفي (Else): تنفذ إذا كان الشرط خاطئاً.

3. كتلة الالتحام (Merge): لكي يكمل البرنامج عمله بسلام في مسار واحد بعد انتهاء الجملة الشرطية.

الخطوة الثانية: تحديث زائر توليد الكود (IRGeneratorVisitor)

افتحوا ملف backend/ir_generator.py الذي نعمل عليه. سنقوم الآن بإضافة الدالة visit_IfNode لبرمجة هذه المناهة المعمارية:

Python

```
def visit_IfNode(self, node):
    # 1. يجب أن ينتج قيمة منطقية (التقييم العميق: حساب قيمة الشرط)
    cond_val = self.visit(node.condition)
    if cond_val is None:
        raise Exception("خطأ هندسي: تعذر حساب قيمة الشرط لجملّة (إذا)")

    # الحصول على الدالة الحالية التي نكتب بداخلها
    current_func = self.builder.function

    # 3. في الذاكرة (Basic Blocks) حجز الكتل الأساسية
    then_bb = current_func.append_basic_block("then")
    merge_bb = current_func.append_basic_block("merge")

    # التحقق مما إذا كان هناك مسار (وإلا)
    if node.else_block:
        else_bb = current_func.append_basic_block("else")
        # else وإلا اذهب لـ then، قفزة مشروطة: إذا صح اذهب لـ
```

```

        self.builder.cbranch(cond_val, then_bb, else_bb)
else:
    # مباشرة merge وإلا تخطى واذهب لـ then، قفزة مشروطة: إذا صح اذهب لـ
    self.builder.cbranch(cond_val, then_bb, merge_bb)

# 4. (then) نقل القلم وبناء كتلة التحقق
self.builder.position_at_end(then_bb)
self.visit(node.then_block) # زيارة محتوى جملة إذا

# برمجة دفاعية: لا نقفز للالتحام إلا إذا كانت الكتلة غير مختومة مسبقاً بقانون
المنهي
if not self.builder.block.is_terminated:
    self.builder.branch(merge_bb)

# 5. إن وجدت (else) نقل القلم وبناء كتلة النفي
if node.else_block:
    self.builder.position_at_end(else_bb)
    self.visit(node.else_block)

# برمجة دفاعية للمسار الثاني
if not self.builder.block.is_terminated:
    self.builder.branch(merge_bb)

# 6. استئناف البرنامج: وضع القلم في كتلة الالتحام ليُكتب الكود القادم فيها
self.builder.position_at_end(merge_bb)

```

الخطوة الثالثة: اختبار المترجم بكود عربي حقيقي

الآن سنعدل برنامجنا الرئيسي لاختبار هذا العقل المدبر الجديد. افتحوا ملف `program.arabic` واكتبوا فيه كوداً يختبر المسارين:

Code-Snippet

```

متغير س : صحيح = ١٠ :
    إذا (س < ٥) {
        س = ١
    } وإلا {
        س = ٠
    }

```

في موجه الأوامر (Terminal)، شغلوا المترجم لتوليد الكود:

Bash

```
python main.py
```

الخطوة الرابعة: الفحص البصري والتدقيق (Debugging)

كمهندسين، نحن لا نكتفي بظهور رسالة النجاح. افتحوا ملف `output.ll` المولد. ستجدون تحفة هندسية تعكس المتاهة التي خططناها بدقة:

Code-Snippet

```

; ModuleID = "arabic_compiler_module"
define i32 @main() {
entry:
    %"س" = alloca i32
    store i32 10, i32* %"س"

```

```

%"_val" = load i32, i32* %"س"
%gttmp = icmp sgt i32 %"_val", 5
br i1 %gttmp, label %then, label %else

then:
    store i32 1, i32* %"س"
    br label %merge

else:
    store i32 0, i32* %"س"
    br label %merge

merge:
    ret i32 0
}

```

قائمة تدقيق المهندس:

- هل الشرط سليم؟ نعم، استخدمنا `icmp sgt` لمقارنة المتغير بالرقم 5.
- هل القفزة المشروطة سليمة؟ نعم، أمر `br i1` يأخذ المسجل المنطقي ويوجه التنفيذ إما لـ `then` أو `else`.
- هل قانون (المنهي) محترم؟ نعم! تأملوا كتلتي `then` و `else`، كلاهما تنتهيان بأمر قفز إجباري `br label %merge` ولا يوجد أي أمر حسابي بعدهما.

أرفع لكم القبة! لقد قمتم للتو ببرمجة أحد أعقد الهياكل في علم المترجمات. استخدامكم لأسلوب البرمجة الدفاعية (`is_terminated`) يثبت أنكم تبنون مترجماً صناعياً قادراً على التعامل مع الأكواد المعقدة والتداخلات دون أن ينهار. في الجلسة القادمة (رقم 17)، سننتقل إلى المستوى التالي: برمجة توليد الـ IR لحلقة "طالما" (`while-loop`) وحل مشكلة العودة لبداية الحلقة.